

# 나를 바꾸는 C++

작성자: [오희성\[Heesung Oh\]](#)

|                                                   |    |
|---------------------------------------------------|----|
| 서문                                                | 3  |
| 1. 왜 C++인가?                                       | 3  |
| 2. 소프트웨어 공학 관점에서의 C++                             | 3  |
| 3. C++을 배우는 방법                                    | 3  |
| 1부. C++ 기초와 객체지향 프로그래밍                            | 5  |
| 01. C++ 언어의 특징과 C에서 C++로                          | 5  |
| 1.1 C++ 언어의 특징                                    | 5  |
| 1.1.1 절차적 프로그래밍과 우수한 성능                           | 5  |
| 1.1.2 객체지향 프로그래밍과 유지보수                            | 6  |
| 1.1.3 제네릭 프로그래밍과 표준 라이브러리                         | 6  |
| 1.1.4 정적 타입과 컴파일 타임 처리                            | 7  |
| 1.1.5 자원 관리와 RAII                                 | 8  |
| 1.1.6 여러 분야에서 사용되는 C++                            | 8  |
| 1.2 C와 C++의 관계                                    | 8  |
| 1.2.1 타입을 이용한 안전한 입출력                             | 9  |
| 1.2.2 C 문법의 계승                                    | 9  |
| 1.2.3 C와 C++의 차이                                  | 10 |
| 1.2.4 C++의 주요 키워드                                 | 11 |
| 1.2.5 C++을 C처럼만 사용할 때의 한계                         | 11 |
| 1.3 C++의 기본형 타입                                   | 12 |
| 1.3.1 기본형 타입의 분류                                  | 12 |
| 1.3.2 정수형과 리터럴 접미사                                | 12 |
| 1.3.3 논리형(bool)                                   | 14 |
| 1.3.4 확장된 문자형과 리터럴 접두사                            | 15 |
| 1.3.5 부동소수점형의 종류와 접미사                             | 15 |
| 1.3.6 void, nullptr, std::nullptr_t               | 16 |
| 1.4 C++에서 추가된 편리한 문법과 기능                          | 17 |
| 1.4.1 new, new[]와 delete, delete[]를 이용한 동적 메모리 할당 | 17 |
| 1.4.1 new, new[]와 delete, delete[]를 이용한 동적 메모리 할당 | 19 |
| 1.4.2 제어문에서 변수 선언하기                               | 21 |
| 1.4.3 범위 기반 for문                                  | 23 |
| 1.4.4 C++ 난수 라이브러리(C++11)                         | 25 |
| 1.5 정리                                            | 26 |
| 1.5 학습 과제                                         | 27 |
| 02. C++ 프로그래밍의 기본                                 | 29 |
| 2.1 이름 공간                                         | 29 |
| 2.1.1 이름 공간과 범위 지정                                | 29 |
| 2.1.2 std 이름 공간과 using 선언                         | 30 |
| 2.1.3 using namespace                             | 30 |
| 2.1.4 중첩 이름 공간과 별칭                                | 30 |
| 2.2 입출력 스트림                                       | 31 |
| 2.2.1 cout과 출력 연산자                                | 31 |
| 2.2.2 줄 바꿈과 출력 버퍼                                 | 32 |
| 2.2.3 출력 형식 지정                                    | 32 |
| 2.2.4 cin과 입력 연산자                                 | 34 |
| 2.2.5 문자열과 getline()                              | 35 |
| 2.2.6 입력 상태와 오류 처리                                | 35 |
| 2.2.7 입력 버퍼 처리                                    | 35 |
| 2.3 타입의 크기와 범위 확인                                 | 36 |
| 2.3.1 데이터 모델과 기본형의 크기                             | 36 |
| 2.3.2 sizeof와 CHAR_BIT                            | 37 |
| 2.3.3 alignof                                     | 37 |
| 2.3.4 std::numeric_limits                         | 38 |
| 2.3.5 std::size_t와 std::ptrdiff_t                 | 38 |
| 2.4 문자형과 문자 인코딩                                   | 39 |
| 2.4.1 char 계열                                     | 39 |
| 2.4.2 와이드 문자와 Unicode 코드 단위                       | 40 |
| 2.4.3 문자열 배열과 널 문자                                | 40 |
| 2.4.4 문자형과 출력 스트림                                 | 41 |
| 2.5 부동소수점형과 오차                                    | 41 |

|                                  |    |
|----------------------------------|----|
| 2.5.1 실수값의 근사 표현                 | 41 |
| 2.5.2 == 비교의 의미                  | 42 |
| 2.5.3 절대 오차와 상대 오차               | 42 |
| 2.5.4 머신 입실론                     | 42 |
| 2.5.5 무한대와 NaN                   | 43 |
| 2.5.6 정확한 값이 필요한 데이터             | 43 |
| 2.6 함수 기능의 확장                    | 44 |
| 2.6.1 함수 오버로딩                    | 44 |
| 2.6.2 기본 인수                      | 45 |
| 2.6.3 인라인 함수                     | 46 |
| 2.6.4 함수의 반환 타입 추론               | 47 |
| 2.7 참조                           | 47 |
| 2.7.1 참조의 개념                     | 47 |
| 2.7.3 상수 참조                      | 49 |
| 2.9 타입 표현과 안전성                   | 53 |
| 2.10 정리                          | 57 |
| 2.4 문자형과 문자 인코딩                  | 61 |
| 2.5 부동소수점형과 오차                   | 62 |
| 2.6 함수 기능의 확장                    | 62 |
| 2.6.1 함수 오버로딩                    | 62 |
| 2.6.2 기본 인수                      | 63 |
| 2.6.3 인라인 함수                     | 63 |
| 2.6.4 함수의 반환 타입 추론               | 64 |
| 2.7 참조                           | 65 |
| 2.7.1 참조의 개념                     | 65 |
| 2.7.2 값, 포인터와 참조 전달              | 65 |
| 2.7.3 상수 참조                      | 66 |
| 2.7.4 참조 반환                      | 67 |
| 2.8 초기화와 <code>const</code>      | 67 |
| 1.7.1 <code>const</code> 와 변경 제한 | 67 |
| 2.9 C++에서 강화된 타입 표현과 안전성         | 68 |
| 2.9.1 <code>const</code> 와 변경 제한 | 68 |

# 서문

## 1. 왜 C++인가?

C++은 배우기 쉬운 언어가 아니다. 문법의 범위가 넓고, 객체의 수명과 메모리, 타입, 성능까지 개발자가 직접 판단해야 하는 부분도 많다. 그럼에도 C++은 오랫동안 시스템 소프트웨어와 다양한 응용 프로그램 개발에 사용되어 왔다.

C++가 계속 사용되는 이유를 단순히 빠른 언어이기 때문이라고만 설명하기는 어렵다. C++은 서로 다른 세 가지 장점을 하나의 언어 안에서 제공한다.

첫째, C에서 이어진 절차적 프로그래밍과 하드웨어에 가까운 제어를 제공한다. 함수와 포인터, 연속된 메모리, 객체의 배치와 수명을 직접 다룰 수 있으며, 컴파일된 코드는 네이티브 환경에서 실행된다. 이러한 특징은 처리 속도와 메모리 사용을 세밀하게 제어할 수 있는 기반이 된다.

둘째, 클래스와 객체지향 프로그래밍을 지원한다. 프로그램의 규모가 커지면 단순히 함수만 나누는 것으로는 데이터의 변경과 기능 사이의 관계를 관리하기 어려워진다. 객체지향 프로그래밍은 상태와 행위를 하나의 객체로 묶고, 프로그램을 역할과 책임에 따라 나눌 수 있게 한다. 이를 통해 변경 범위를 제한하고, 여러 기능이 연결된 프로그램을 더 체계적으로 관리할 수 있다.

셋째, 방대한 표준 라이브러리를 제공한다. 동적 배열, 연결 리스트, 트리, 해시 테이블과 같은 자료구조를 매번 직접 구현할 필요가 없으며, 검색과 정렬, 변환, 파일 입출력, 시간, 난수, 메모리 관리 등 다양한 기능을 검증된 형태로 사용할 수 있다. 개발자는 자료구조의 원리를 이해하면서도 실제 구현에서는 표준 컨테이너와 알고리즘을 활용해 응용 프로그램의 핵심 기능에 집중할 수 있다.

C++은 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍을 함께 지원하는 언어다. 모든 코드를 반드시 클래스로 작성할 필요도 없고, 성능을 위해 모든 추상화를 포기할 필요도 없다. 문제에 따라 적절한 방법을 선택하고 여러 방식을 조합할 수 있다는 점이 C++의 중요한 특징이다.

이러한 이유로 C++은 운영체제와 시스템 소프트웨어뿐 아니라 그래픽스, 게임 엔진, 시뮬레이션, 로봇, 임베디드 시스템, 금융, 데이터 처리, 네트워크 서버, 데스크톱 응용 프로그램과 개발 도구 등 여러 분야에서 사용되고 있다. 특히 높은 성능과 복잡한 프로그램 구조를 함께 다루어야 하는 응용 분야에서 C++의 장점이 잘 드러난다.

C++은 과거의 언어에 머물러 있지 않다. C++11 이후 이동 의미론, 스마트 포인터, 람다, `constexpr`, `Ranges`, `Concepts`와 같은 기능이 추가되면서 객체의 수명과 소유권을 더 명확하게 표현하고, 표준 라이브러리를 더 안전하고 편리하게 활용할 수 있도록 발전해 왔다. 현대 C++은 기존의 저수준 제어 능력을 유지하면서 더 높은 수준의 추상화를 함께 제공하는 방향으로 계속 변화하고 있다.

## 2. 소프트웨어 공학 관점에서의 C++

프로그램은 처음 작성할 때보다 기능이 추가되고 수정되는 과정에서 더 복잡해진다. 여러 기능이 하나의 데이터에 의존하고, 객체의 생성과 제거 시점이 서로 얽히면, 한 부분의 변경이 예상하지 못한 곳까지 영향을 미치기도 한다. 따라서 소프트웨어 개발에서는 기능의 구현뿐 아니라 복잡성과 변경 비용을 관리하는 것이 중요하다.

C++은 이러한 문제를 언어의 구조와 직접 연결한다. 클래스는 데이터와 행위를 하나의 책임 단위로 묶고, 접근 제어는 객체의 내부 상태가 임의로 변경되는 것을 제한한다. 상속과 다형성은 공통 인터페이스를 통해 여러 구현을 다룰 수 있게 하고, 템플릿은 타입에 의존하지 않는 일반화된 코드를 만들 수 있게 한다.

또한 C++에서는 객체의 수명과 자원의 소유권이 중요한 설계 요소가 된다. 동적으로 할당한 메모리뿐 아니라 파일, 네트워크 연결, 잠금, 그래픽 자원과 같은 모든 자원에는 획득과 해제 시점이 존재한다. C++의 **RAII**는 자원을 객체의 생명주기에 연결하여 객체가 생성될 때 자원을 획득하고, 객체가 소멸할 때 자동으로 해제하도록 한다. 스마트포인터는 단순히 포인터를 편리하게 사용하는 도구가 아니라, 누가 객체를 소유하고 언제 제거할지를 코드로 표현하는 수단이다.

C++은 추상화의 비용도 비교적 직접적으로 드러낸다. 가상 함수는 런타임 다형성을 제공하지만 간접 호출이 발생할 수 있고, 동적 메모리 할당은 유연성을 제공하지만 할당과 해제 비용을 가진다. 템플릿은 런타임 비용을 줄일 수 있지만 컴파일 시간과 코드 크기에 영향을 줄 수 있다. 개발자는 각 기능의 장점만이 아니라 그 기능이 요구하는 비용도 함께 고려해야 한다.

이러한 관점은 C++의 제로 오버헤드 원칙과도 연결된다. 사용하지 않는 기능에는 비용을 지불하지 않으며, 추상화를 사용하더라도 직접 작성한 저수준 코드와 비슷한 수준의 성능을 목표로 한다. C++에서 추상화와 성능은 서로 반대되는 개념이 아니라, 함께 고려해야 하는 설계 요소다.

그러나 클래스와 상속을 사용한다고 해서 프로그램의 유지보수성이 자동으로 좋아지는 것은 아니다. 책임이 지나치게 많은 클래스, 깊은 상속 구조, 불필요한 추상화는 오히려 코드를 복잡하게 만든다. 중요한 것은 기능을 많이 사용하는 것이 아니라, 상태와 책임, 수명과 소유권, 객체 사이의 관계를 명확하게 표현하는 것이다.

이 책에서는 C++ 문법을 개별 기능의 목록으로만 설명하지 않는다. 각 기능이 객체의 상태와 생명주기, 자원 관리, 타입 안전성, 표준 라이브러리와 어떤 관계를 가지는지 함께 살펴본다.

### 3. C++을 배우는 방법

이 책은 C언어의 기본 문법을 학습한 독자를 대상으로 한다. 변수와 연산자, 조건문과 반복문, 함수, 배열, 구조체와 포인터를 처음부터 다시 설명하지 않는다. 대신 C를 사용하던 방식에서 C++의 객체 모델과 표준 라이브러리를 활용하는 방식으로 어떻게 전환해야 하는지를 중심으로 다룬다.

먼저 C와 C++의 차이를 살펴보고 C++에서 추가된 기본형 타입, 이름 공간, 스트림 입출력, 함수 오버로딩, 기본 인수와 참조를 정리한다. 이후 구조체에 데이터만 저장하던 방식에서 벗어나, 상태와 행위를 함께 관리하는 클래스와 객체의 구조로 넘어간다.

객체지향 프로그래밍을 학습할 때는 클래스 문법만 익혀서는 충분하지 않다. 객체가 언제 생성되고 소멸하는지, 복사할 때 어떤 의미를 가지는지, 자원을 다른 객체로 이동할 때 소유권이 어떻게 변하는지를 이해해야 한다. 생성자와 소멸자, 복사와 이동, RAII와 스마트 포인터는 서로 분리된 문법이 아니라 객체의 생명주기를 표현하는 하나의 흐름이다.

상속과 다형성도 가상 함수의 사용법만을 배우는 데 그치지 않는다. 객체 사이의 관계를 어떻게 정할 것인지, 상속과 구성 중 무엇을 선택할 것인지, 인터페이스를 어느 범위까지 제공할 것인지 판단해야 한다. SOLID 원칙은 별도의 이론으로 외우기보다 이러한 객체지향 구조를 점검하는 기준으로 활용한다.

템플릿과 제네릭 프로그래밍을 익힌 뒤에는 표준 라이브러리로 범위를 확장한다. 컨테이너의 함수 이름을 외우는 것보다 각 컨테이너가 데이터를 어떤 방식으로 저장하며, 접근과 삽입, 삭제에 어떤 비용이 필요한지를 이해하는 것이 중요하다. 반복자, 알고리즘, 함수 객체와 람다는 컨테이너에 저장된 데이터를 일관된 방식으로 처리하기 위한 구조로 연결해서 학습한다.

현대 C++ 기능은 별도의 문법 묶음으로 분리하지 않는다. auto는 타입을 표현하는 위치에서, constexpr은 초기화와 컴파일 타임 평가에서, 이동 의미론은 복사와 자원 이전에서, 스마트 포인터는 소유권에서, 람다와 Ranges는 알고리즘을 다루는 과정에서 설명한다. 새로운 문법은 그 기능이 필요한 상황과 함께 학습해야 의미를 제대로 이해할 수 있기 때문이다.

마지막으로 대표적인 디자인 패턴을 통해 앞에서 배운 내용을 실제 객체 구조로 연결한다. 싱글턴, 팩토리 메서드, 추상 팩토리, 어댑터, 컴포지트, 퍼사드, 옵저버와 커맨드 패턴을 다루며, 패턴의 클래스 구조를 암기하기보다 어떤 문제를 해결하기 위해 사용되는지 살펴본다. 특히 커맨드 패턴에서는 요청을 객체로 표현하고 실행 기록을 저장하여 Undo와 Redo를 구성하는 과정을 통해 객체지향, 다형성, 스마트 포인터와 표준 컨테이너가 실제 구조에서 어떻게 결합되는지 확인한다.

C++은 문법을 많이 아는 것으로 완성되는 언어가 아니다. 문법은 프로그램의 구조를 표현하기 위한 수단이다. 중요한 것은 어떤 객체가 상태를 소유하는지, 누가 자원을 관리하는지, 변경되는 부분을 어떻게 분리하는지, 어떤 자료구조와 알고리즘을 선택할지를 판단하는 능력이다.

C++을 배우는 과정은 단순히 새로운 문법을 추가하는 과정이 아니다. 프로그램을 구성하고 자원과 비용을 바라보는 방식을 바꾸는 과정이다. 이 책의 제목을 \*\*《나를 바꾸는 C++》\*\*이라고 정한 이유도 여기에 있다.

# 1부. C++ 기초와 객체지향 프로그래밍

## 01. C++ 언어의 특징과 C에서 C++로

C++는 한 가지 프로그래밍 방식만을 강요하는 언어가 아니다. C에서 이어진 절차적 프로그래밍을 사용할 수 있고, 클래스와 객체를 중심으로 프로그램을 구성할 수도 있으며, 템플릿과 표준 라이브러리를 이용해 다양한 자료구조와 알고리즘을 재사용할 수도 있다.

C++가 오랫동안 여러 분야에서 사용되는 이유는 이러한 특징이 함께 존재하기 때문이다. 하드웨어와 가까운 수준에서 프로그램을 제어할 수 있어 높은 성능을 얻을 수 있고, 객체지향 프로그래밍을 통해 규모가 커진 프로그램을 역할과 책임에 따라 나눌 수 있으며, 표준 라이브러리에 포함된 다양한 기능을 이용해 이미 검증된 자료구조와 알고리즘을 활용할 수 있다. 또한 정적 타입 검사와 타입 추론, 컴파일 타임 계산을 통해 안정성과 성능을 함께 고려할 수 있다.

이러한 특징으로 C++는 운영체제와 시스템 소프트웨어뿐 아니라 그래픽스, 게임 엔진, 시뮬레이션, 로봇, 임베디드 시스템, 금융, 데이터 처리, 데스크톱 응용 프로그램 등 성능과 복잡한 구조를 함께 다루어야 하는 분야에서 널리 사용된다.

이 장에서는 C++의 특징을 먼저 살펴본 뒤, C를 학습한 개발자가 C++로 넘어갈 때 알아야 할 언어적 차이를 정리한다. C 문법을 처음부터 반복하지 않고, C++에서 추가되거나 의미가 달라진 기능을 중심으로 설명한다.

### 1.1 C++ 언어의 특징

C++는 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍을 함께 지원하는 다중 패러다임 언어이다. 프로그램의 모든 부분을 반드시 클래스로 작성할 필요는 없으며, 문제의 성격에 따라 적절한 방식을 선택하거나 여러 방식을 함께 사용할 수 있다.

C++의 주요 특징은 다음 네 가지 관점으로 정리할 수 있다.

- 절차적 프로그래밍과 하드웨어에 가까운 제어를 통한 성능
- 객체지향 프로그래밍을 통한 구조화와 유지보수
- 방대한 표준 라이브러리를 통한 자료구조와 알고리즘의 재사용
- 정적 타입 검사와 컴파일 타임 처리를 통한 안정성과 성능

#### 1.1.1 절차적 프로그래밍과 우수한 성능

C++는 C의 문법과 실행 모델을 상당 부분 계승했다. 변수, 조건문, 반복문, 함수, 배열, 구조체, 포인터를 이용해 절차적으로 프로그램을 작성할 수 있다. 작은 함수들을 조합하여 처리 과정을 명확하게 표현하는 방식은 지금도 유효하며, 간단하거나 성능이 중요한 코드에서 널리 사용된다.

다만 절차적 프로그래밍 자체가 자동으로 높은 성능을 만드는 것은 아니다. C++가 높은 성능을 얻을 수 있는 이유는 다음과 같은 특성을 제공하기 때문이다.

- 컴파일된 코드의 네이티브 기계어 실행
- 메모리 배치와 객체 수명의 직접 제어
- 포인터와 참조를 이용한 데이터 직접 접근
- 목적에 따른 스택과 동적 메모리 선택
- 사용하지 않는 기능에 대한 비용 배제
- 템플릿 기반 추상화의 컴파일 단계 최적화

C++에서는 고수준의 표현을 사용하더라도 불필요한 런타임 비용을 줄일 수 있다. 이를 흔히 제로 오버헤드 원칙(**zero-overhead principle**)이라고 한다. 사용하지 않는 기능에는 비용이 발생하지 않으며, 사용하는 추상화는 같은 기능을 직접 작성한 저수준 코드와 비슷한 수준의 성능을 목표로 한다.

다음 함수는 포인터와 원소 수를 직접 전달받아 정수 배열의 합계를 계산한다.

```
int SumLowLevel(const int* values, int count)
{
    int result = 0;
    for (int i = 0; i < count; ++i)
    {
        result += values[i];
    }
    return result;
}
```

같은 기능을 C++의 `std::span`과 범위 기반 `for` 문을 사용하여 다음과 같이 표현할 수 있다.

```
#include <span>
int Sum(std::span<const int> values)
{
    int result{};
    for (int value : values)
    {
        result += value;
    }
}
```

```

    }
    return result;
}

```

두 번째 함수는 배열의 시작 주소와 원소 수를 `std::span`이라는 하나의 범위로 묶어 전달한다. 배열의 원소를 복사하지 않으며, 범위 기반 `for` 문도 내부적으로 각 원소를 순서대로 접근하는 반복 코드로 변환된다.

```

#include <iostream>
#include <span>
int SumLowLevel(const int* values, int count);
int Sum(std::span<const int> values);
int main()
{
    int values[]{ 10, 20, 30, 40 };
    int lowLevelResult{ SumLowLevel(values, 4) };
    int spanResult{ Sum(values) };

    std::cout << "포인터와 원소 수: " << lowLevelResult << '\n';
    std::cout << "std::span: " << spanResult << '\n';
}
// SumLowLevel, Sum 함수 정의
// ...

```

`Sum` 함수는 포인터와 원소 수를 따로 전달하는 함수보다 호출 방법이 명확하고, 원소 수를 잘못 전달할 가능성도 줄어든다. 그러나 `std::span`은 일반적으로 시작 주소와 원소 수만 저장하는 가벼운 객체이며, 컴파일러가 최적화하면 두 함수는 비슷한 형태의 기계어로 변환될 수 있다.

즉, 개발자는 더 안전하고 읽기 쉬운 표현을 사용하면서도 불필요한 실행 시간 비용을 추가하지 않을 수 있다. 이것이 **C++**에서 말하는 제로 오버헤드 원칙의 대표적인 예이다.

**C++**가 객체지향 프로그래밍을 지원한다고 해서 모든 연산을 클래스에 넣어야 하는 것은 아니다. 단순한 계산은 일반 함수로 작성하고, 데이터와 동작을 하나의 개념으로 묶어야 할 때는 클래스를 사용하는 등 문제에 적합한 구조를 선택하는 것이 중요하다.

### 1.1.2 객체지향 프로그래밍과 유지보수

프로그램의 규모가 커지면 단순히 함수를 나누는 것만으로는 전체 구조를 관리하기 어려워진다. 여러 함수가 같은 데이터를 공유하고, 데이터의 변경 규칙이 여러 위치에 흩어지면 한 부분을 수정했을 때 다른 기능까지 영향을 받을 수 있다.

객체지향 프로그래밍은 데이터와 그 데이터를 처리하는 기능을 하나의 객체 안에 묶는다. **C++**에서는 클래스를 이용하여 객체의 상태와 행위를 함께 정의할 수 있다.

```

class Character
{
public:
    void TakeDamage(int damage)
    {
        hp -= damage;
        if (hp < 0)
            hp = 0;
    }
    int GetHp() const
    {
        return hp;
    }
private:
    int hp = 100;
};

```

이 클래스는 체력이라는 상태와 피해를 처리하는 행위를 함께 관리한다. 외부 코드가 `hp` 값을 임의로 변경하지 못하도록 제한하고, `TakeDamage()`를 통해서만 상태가 변경되도록 구성했다.

이와 같은 구조는 다음과 같은 장점을 제공한다.

- 데이터 변경 범위 제한
- 객체의 유효 상태 유지
- 관련 데이터와 기능 통합
- 역할과 책임에 따른 구조 분리
- 내부 구현 변경 영향 최소화
- 기능별 개발 분담과 협업

객체지향 프로그래밍을 사용한다고 해서 유지보수성이 자동으로 좋아지는 것은 아니다. 책임이 지나치게 많은 클래스, 깊은 상속 구조, 불필요한 추상화는 오히려 프로그램을 복잡하게 만들 수 있다. 객체지향의 목적은 클래스를 많이 만드는 것이 아니라, 변경될 가능성이 있는 부분을 분리하고 객체 사이의 책임을 명확하게 만드는 데 있다.



### 1.1.3 제네릭 프로그래밍과 표준 라이브러리

C++ 표준 라이브러리는 문자열, 자료구조, 알고리즘, 파일 입출력, 시간, 난수, 메모리 관리 등 프로그램 작성에 필요한 다양한 기능을 제공한다.

C에서 여러 개의 데이터를 관리하려면 배열을 직접 할당하고 크기를 별도로 저장하거나, 연결 리스트와 트리 같은 자료구조를 직접 구현해야 하는 경우가 많았다. C++에서는 표준 라이브러리의 컨테이너를 이용해 이미 구현되고 검증된 자료구조를 사용할 수 있다.

대표적인 컨테이너는 다음과 같다.

- `std::array`: 크기가 고정된 배열
- `std::vector`: 크기가 변하는 연속 배열
- `std::deque`: 앞과 뒤에서 원소를 추가하거나 제거할 수 있는 컨테이너
- `std::list`: 연결 리스트
- `std::map`: 정렬된 키와 값
- `std::set`: 중복되지 않는 정렬된 값
- `std::unordered_map`: 해시를 이용한 키와 값
- `std::string`: 문자열

표준 라이브러리는 자료구조만 제공하는 것이 아니다. 컨테이너의 원소를 검색하고 정렬하고 변환하는 알고리즘도 함께 제공한다.

```
#include <algorithm>
#include <iostream>
#include <vector>
int main()
{
    std::vector<int> values = { 30, 10, 50, 20, 40 };
    std::sort(values.begin(), values.end());
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

`std::vector`는 데이터 저장을 담당하고, `std::sort`는 정렬을 담당한다. 개발자가 동적 배열과 정렬 알고리즘을 매번 직접 구현할 필요가 없다. 표준 라이브러리의 장점은 단순히 코드가 짧아지는 데 있지 않다.

- 여러 환경에서 널리 사용되고 검증된 구현 활용
- 컨테이너와 알고리즘의 일관된 구성
- 템플릿을 통한 다양한 타입의 재사용
- 타입에 맞는 코드의 컴파일 단계 최적화
- 자료구조 구현에 필요한 시간과 오류 감소

자료구조의 원리와 시간 복잡도는 여전히 이해해야 한다. 그러나 실제 프로그램에서 연결 리스트, 동적 배열, 해시 테이블을 매번 처음부터 구현할 필요는 없다. 개발자는 문제에 적합한 컨테이너를 선택하고, 데이터의 저장 방식과 탐색 비용을 판단하는 데 집중할 수 있다.

### 1.1.4 정적 타입과 컴파일 타임 처리

C++는 변수와 표현식의 타입이 컴파일 시점에 결정되는 정적 타입 언어이다. 컴파일러는 서로 대입할 수 없는 타입, 잘못된 함수 호출, 조건을 만족하지 않는 템플릿 사용 등을 프로그램이 실행되기 전에 검사한다.

C++는 C의 기본적인 타입 체계를 계승하면서도 잘못된 사용을 줄이기 위한 기능을 추가하였다. `nullptr`는 정수값 0과 널 포인터를 구분하고, `enum class`는 열거형 값의 범위를 제한한다. 또한 C++ 형 변환 연산자를 사용하면 어떤 종류의 변환을 수행하는지 코드에 명확하게 표현할 수 있다.

C++는 정적 타입 언어이지만 모든 타입을 직접 작성할 필요는 없다. `auto`와 `decltype`을 사용하면 컴파일러가 초기값이나 표현식을 분석하여 타입을 결정한다. 타입 추론은 타입을 없애는 기능이 아니다. 컴파일러가 타입을 대신 결정하는 것이며, 결정된 타입은 실행 중에 변경되지 않는다.

C++는 일부 계산과 검사도 컴파일 시점에 수행할 수 있다. `constexpr`는 값이나 함수를 상수 표현식으로 사용할 수 있게 하고, `static_assert`는 지정한 조건이 만족되지 않으면 컴파일을 중단한다.

```
constexpr int Square(int value)
{
    return value * value;
}
constexpr int size = Square(10);
static_assert(size == 100);
```

`constexpr` 함수의 모든 호출이 반드시 컴파일 시점에 계산되는 것은 아니다. 상수 표현식이 필요한 상황에서는 컴파일 시점에 평가되지만, 실행 중에 결정되는 값을 전달하면 일반 함수처럼 실행 시점에 계산될 수도 있다. 반드시 컴파일 시점에 평가해야 하는 함수에는 `constexpr`를 사용한다.



이러한 기능은 잘못된 타입 사용을 실행 전에 발견하고, 일부 계산을 컴파일 과정으로 옮겨 실행 시간의 비용을 줄일 수 있게 한다. **C++**는 하드웨어에 가까운 제어 능력을 유지하면서도 타입 검사와 컴파일 타임 기능을 통해 안정성과 성능을 함께 고려할 수 있는 언어이다.

**nullptr**, **enum class**, **auto**, **decltype**과 형 변환 연산자의 구체적인 사용법은 1.7절에서 다시 살펴본다. **constexpr**, **constexpr**, **constexpr**은 객체 초기화와 컴파일 타임 평가를 다루는 단원에서 자세히 설명한다.

### 1.1.5 자원 관리와 RAII

프로그램은 메모리뿐 아니라 파일, 네트워크 연결, 잠금, 그래픽스 자원 등 다양한 자원을 사용한다. 이러한 자원은 사용이 끝난 시점에 정확하게 반환해야 한다.

**C++**에서는 객체의 생성과 소멸을 자원 관리에 연결하는 **RAII(Resource Acquisition Is Initialization)** 방식을 널리 사용한다. 객체가 생성될 때 자원을 확보하고, 객체의 수명이 끝날 때 소멸자를 통해 자원을 자동으로 반환한다.

```
#include <memory>
void ProcessData(bool cancel)
{
    auto values = std::make_unique<int[]>(100);
    values[0] = 10;
    values[1] = 20;
    if (cancel)
    {
        return;
    }
    values[2] = values[0] + values[1];
}
```

**std::make\_unique<int[]>(100)**은 정수 100개를 저장할 동적 메모리를 확보하고, 그 메모리의 소유권을 **std::unique\_ptr** 객체에 전달한다. **auto**는 초기값을 분석하여 **values**의 타입을 자동으로 결정한다.

**values**가 함수 범위를 벗어나면 **std::unique\_ptr**의 소멸자가 호출되어 동적 메모리를 자동으로 해제한다. 함수가 정상적으로 끝나는 경우뿐 아니라 중간에 **return**이 실행되거나 예외가 발생하는 경우에도 객체의 소멸 과정은 유지된다. 따라서 개발자가 **delete[]**를 직접 호출할 필요가 없다.

**RAII**는 다음과 같은 자원 관리에 활용된다.

- 동적 메모리의 자동 해제
- 파일과 네트워크 연결의 종료
- 잠금의 자동 해제
- 운영체제 및 그래픽스 자원의 반환
- 예외 발생 시 자원 누수 방지

**std::vector**, **std::string**, 스마트 포인터와 같은 표준 라이브러리 타입도 **RAII**를 기반으로 자원을 관리한다. 개발자는 자원의 해제 시점을 일일이 추적하기보다 객체의 수명과 소유 관계를 설계하는 데 집중할 수 있다.

### 1.1.6 여러 분야에서 사용되는 C++

**C++**는 높은 성능만 필요한 언어도 아니고, 객체지향만을 위한 언어도 아니다. 성능, 메모리 제어, 복잡한 프로그램 구조, 재사용 가능한 라이브러리가 함께 필요한 분야에서 장점을 가진다.

대표적인 활용 분야는 다음과 같다.

- 운영체제와 시스템 소프트웨어
- 게임 엔진과 실시간 콘텐츠
- 그래픽스와 렌더링
- 물리 및 과학 시뮬레이션
- 로봇과 자율주행 시스템
- 임베디드 시스템
- 금융 및 대규모 데이터 처리
- 데이터베이스와 네트워크 서버
- 웹 브라우저와 데스크톱 응용 프로그램
- 개발 도구와 콘텐츠 제작 도구

특히 응용 프로그램은 성능만으로 완성되지 않는다. 사용자 기능이 계속 추가되고, 데이터 구조가 변하며, 여러 시스템이 서로 연결된다.

**C++**는 절차적 코드의 직접성과 객체지향 구조의 관리 능력, 표준 라이브러리의 재사용성, 컴파일 타임 처리 능력을 함께 제공하므로 복잡한 응용 프로그램을 만드는 데 적합하다.



## 1.2 C와 C++의 관계

C++는 C 언어를 기반으로 발전했으며, 변수와 연산자, 제어문(조건문, 반복문), 함수, 배열, 구조체 및 포인터 등 C의 핵심 문법과 표준 라이브러리를 상당 부분 계승했다. 따라서 C 언어를 이미 학습한 독자라면 기존 지식을 바탕으로 더욱 수월하게 C++를 익힐 수 있다.

다만, C++가 단순히 C 언어에 클래스(Class) 문법만 추가한 것은 아니다. 보다 엄격해진 타입 검사(Type Checking)와 확장된 함수 문법을 비롯해 참조(Reference), 이름공간(Namespace), 템플릿(Template), 그리고 향상된 표준 라이브러리(STL) 등 프로그램을 더 안전하고 체계적으로 구성할 수 있는 강력한 기능들이 대거 추가되었다.

이 절에서는 앞으로 다룰 예제들의 입력과 실행 결과를 확인하기 위해 필요한 기본 입출력 방식을 먼저 살펴본 뒤, C 언어에서 그대로 이어지는 부분과 C++에서 새롭게 달라진 특징들을 차례로 정리한다.

### 1.2.1 타입을 이용한 안전한 입출력

C에서는 값을 입력하고 출력할 때 주로 `scanf()`와 `printf()`를 사용한다.

```
int level = 0;
double speed = 0.0;
scanf("%d %lf", &level, &speed);
printf("Level: %d, Speed: %.1f\n", level, speed);
```

`scanf()`는 값을 저장할 변수의 주소를 전달해야 하므로 일반 변수 앞에 주소 연산자 `&`를 붙인다. 또한 `%d`, `%f`, `%lf`와 같은 형식 지정자를 변수의 타입에 맞게 작성해야 한다. 주소 연산자를 빠뜨리거나 형식 지정자와 변수의 타입을 잘못 연결하면 예상하지 못한 결과가 발생할 수 있다.

C++에서는 표준 입력 객체인 `cin`과 표준 출력 객체인 `cout`을 사용할 수 있다. 두 객체는 C++ 표준 라이브러리의 `std` 이름 공간에 정의되어 있다.

```
#include <iostream>
int main()
{
    using std::cin;
    using std::cout;

    int level = 0;
    double speed = 0.0;
    cout << "레벨과 이동 속도를 입력하세요: ";
    cin >> level >> speed;
    cout << "Level: " << level
         << ", Speed: " << speed << '\n';
}
```

`cin`과 `cout`은 변수와 표현식의 타입에 따라 적절한 입출력 동작을 선택한다. 따라서 형식 지정자를 직접 작성하지 않으며, `cin`으로 일반 변수에 값을 입력할 때 주소 연산자 `&`도 사용하지 않는다.

```
cin >> level;
cout << level << '\n';
```

공백이 포함된 한 줄의 문자열은 `std::string`과 `getline()`을 이용해 입력할 수 있다.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name;
    cout << "이름을 입력하세요: ";
    getline(cin, name);
    cout << "입력한 이름: " << name << '\n';
}
```

C++의 입출력도 잘못된 입력이나 입력 버퍼와 같은 문제를 별도로 처리해야 한다. 그러나 변수의 주소와 형식 지정자를 개발자가 직접 맞춰야 하는 부담을 줄이고, 타입에 맞는 입출력 동작을 선택함으로써 단순한 실수의 가능성을 낮춘다.

이 장부터 예제의 입력과 출력에는 `scanf()`와 `printf()` 대신 `cin`, `cout`, `getline()`을 사용한다. 이름 공간과 입출력 스트림의 구조, 공백을 포함한 문자열 입력, 입력 오류와 버퍼 처리, 출력 형식 지정은 입출력 단원에서 자세히 살펴본다.

### 1.2.2 C 문법의 계승

C++는 C의 문법과 실행 방식을 상당 부분 계승하였다. C에서 사용한 다음 개념은 C++에서도 그대로 활용할 수 있다.

- 기본형 타입과 변수
- 산술·관계·논리 연산자
- 조건문과 반복문
- 함수
- 배열과 구조체
- 포인터와 함수 포인터
- 동적 메모리
- 파일 입출력

예를 들어 다음과 같은 C의 구조체와 함수 작성 방식은 C++에서도 사용할 수 있다.

```
struct Item
{
    char name[32];
};
struct Character
{
    int hp;
    int attack;
    struct Item* pItem;
};
void TakeDamage(struct Character* character, int damage)
{
    character->hp -= damage;
    if (character->hp < 0)
    {
        character->hp = 0;
    }
}
```

C++에서는 구조체 이름 자체가 타입 이름이므로 **struct**를 반복하지 않아도 된다.

```
struct Character
{
    // ...
    Item* pItem;
};
void TakeDamage(Character* character, int damage)
{
    // ...
}
```

C를 학습한 독자는 함수, 배열, 구조체와 포인터에 관한 기존 지식을 그대로 활용할 수 있다. 그 위에 참조, 클래스, 템플릿과 표준 라이브러리 등의 기능을 단계적으로 더하면 된다.

다만 C에서 사용하던 모든 작성 방식이 C++에서도 가장 적절한 것은 아니다. C++에서는 같은 기능을 더 안전하고 명확하게 표현할 수 있는 문법과 라이브러리를 제공하므로 기존 C 문법을 바탕으로 C++에 적합한 작성 방식으로 확장할 필요가 있다.

### 1.2.3 C와 C++의 차이

C++는 C에 다음과 같은 기능을 추가하거나 확장하였다.

- 이름 공간
- 참조
- 함수 오버로딩과 기본 인수
- 클래스와 접근 제어
- 생성자와 소멸자
- 상속과 다형성
- 예외 처리
- 템플릿과 템플릿 인수 추론
- 표준 라이브러리
- RAII와 스마트 포인터
- **auto**와 **decltype**
- **nullptr**와 **enum class**
- 람다 표현식과 **Ranges**

C++는 타입 검사에서도 C보다 엄격한 부분이 있다. 예를 들어 C에서는 `void*`를 다른 포인터 타입에 암시적으로 대입할 수 있지만, C++에서는 명시적인 형 변환이 필요하다.

```
void* memory = nullptr;
// int* pointer = memory; // C++에서는 오류
int* pointer = static_cast<int*>(memory);
```

C++에서는 함수 오버로딩과 템플릿처럼 타입에 따라 동작이 달라지는 기능을 제공하므로, 타입을 더 명확하게 구분해야 한다. 이 때문에 C에서 허용되는 코드가 C++에서는 허용되지 않거나 의미가 달라지는 경우가 있다. 따라서 C++는 C의 문법을 대부분 계승하지만 C의 모든 프로그램을 그대로 받아들이는 완전한 상위 호환 언어는 아니다.

1.2.4 C++의 주요 키워드

키워드(keyword)는 언어의 문법에서 특별한 의미로 사용하도록 예약된 단어이다. 키워드는 변수, 함수, 구조체와 클래스의 이름으로 사용할 수 없다.

```
// int class = 10;      // 오류: class는 키워드
// void return();      // 오류: return은 키워드
```

C++는 C의 키워드를 대부분 계승하면서 객체지향, 템플릿, 예외 처리와 타입 안전성을 표현하기 위한 키워드를 추가하였다. C++ 코드를 읽을 때 자주 만나는 주요 키워드는 다음과 같다.

| 구분         | 주요 키워드                                                                                                         |
|------------|----------------------------------------------------------------------------------------------------------------|
| 논리값과 널 포인터 | <code>bool</code> , <code>true</code> , <code>false</code> , <code>nullptr</code>                              |
| 이름과 타입 정의  | <code>namespace</code> , <code>using</code> , <code>class</code> , <code>typename</code>                       |
| 객체와 접근 제어  | <code>public</code> , <code>protected</code> , <code>private</code> , <code>this</code> , <code>friend</code>  |
| 상속과 다형성    | <code>virtual</code> , <code>override</code> , <code>final</code>                                              |
| 템플릿과 타입 추론 | <code>template</code> , <code>auto</code> , <code>decltype</code>                                              |
| 형 변환       | <code>static_cast</code> , <code>const_cast</code> , <code>reinterpret_cast</code> , <code>dynamic_cast</code> |
| 예외 처리      | <code>try</code> , <code>catch</code> , <code>throw</code> , <code>noexcept</code>                             |
| 컴파일 타임 처리  | <code>constexpr</code> , <code>constexpr</code> , <code>constexpr</code> , <code>static_assert</code>          |
| 메모리와 연산자   | <code>new</code> , <code>delete</code> , <code>operator</code> , <code>explicit</code>                         |

이 표는 C++의 모든 키워드를 나열한 것이 아니라 이후 단원에서 자주 사용할 키워드를 기능별로 정리한 것이다. 각 키워드의 문법과 사용 목적은 관련 단원에서 자세히 살펴본다.

1.2.5 C++을 C처럼만 사용할 때의 한계

C++에서도 C 방식으로 배열을 만들고 `malloc()`과 `free()`로 메모리를 관리하며 구조체와 전역 함수를 중심으로 프로그램을 작성할 수 있다. 규모가 작고 동작이 단순한 프로그램에서는 이러한 방식도 사용할 수 있다. 그러나 프로그램의 기능과 데이터가 늘어나면 다음과 같은 문제를 개발자가 직접 관리해야 한다.

- 할당된 메모리를 해제하지 않아 발생하는 메모리 누수
- 이미 해제되었거나 수명이 끝난 대상을 가리키는 댕글링 포인터
- 배열 범위를 벗어난 접근
- 여러 함수가 같은 데이터를 변경하면서 발생하는 상태 관리 문제
- 자원의 소유자와 해제 책임이 불분명하여 발생하는 중복 해제
- 한 부분의 수정이 다른 부분에 미치는 예상하지 못한 영향

다음 코드는 동적 배열을 C 방식으로 관리하는 예이다.

```
#include <cstdlib>
void ProcessData()
{
    int* values = static_cast<int*>(std::malloc(sizeof(int) * 100));
    if (values == nullptr)
    {
```

```
        return;
    }
    // 데이터 사용
    // ...
    std::free(values);
}
```

이 코드에서는 메모리의 소유자와 해제 위치를 개발자가 직접 관리한다. 함수가 중간에 끝나거나 예외가 발생하여 `std::free()`에 도달하지 못하면 메모리 누수가 발생할 수 있다.

C++에서는 같은 목적을 표준 컨테이너로 표현할 수 있다.

```
#include <vector>
void ProcessData()
{
    std::vector<int> values(100);
    // 데이터 사용
    // ...
}
```

`std::vector`는 저장 공간과 원소 수를 하나의 객체로 관리한다. `values`의 수명이 끝나면 내부 메모리가 자동으로 해제되므로 해제 위치를 직접 추적할 필요가 없다.

C 방식과 C++ 방식의 차이는 단순히 문법이 짧아지는 데 있지 않다. 데이터의 변경 규칙, 자원의 수명과 소유 관계를 프로그램의 구조 안에 표현하는 방식이 달라진다. C++를 사용한다고 모든 오류가 자동으로 사라지는 것은 아니다. 그러나 클래스, RAII, 표준 컨테이너, 스마트 포인터와 타입 시스템을 적절히 활용하면 개발자가 직접 추적해야 하는 상태와 자원 관리의 범위를 줄일 수 있다.

### 1.3 C++의 기본형 타입

C++의 타입은 크게 기본형 타입, 복합 타입과 사용자 정의 타입으로 나눌 수 있다. 기본형 타입은 언어가 직접 제공하는 타입이다. 복합 타입에는 포인터, 참조, 배열과 함수 타입 등이 포함된다. 클래스, 구조체, 공용체와 열거형은 사용자 정의 타입에 해당한다.

C++는 C의 기본형 타입을 대부분 그대로 사용하면서 논리형과 Unicode 문자형, 널 포인터를 나타내는 타입 등을 추가하였다. 이 절에서는 C와 같은 부분을 반복하기보다 C++에서 추가되거나 주의해야 할 부분을 중심으로 살펴본다.

#### 1.3.1 기본형 타입의 분류

C++의 기본형 타입은 다음과 같이 분류할 수 있다.

| 분류       | 타입                                                                     |
|----------|------------------------------------------------------------------------|
| 논리형      | bool                                                                   |
| 문자형      | char, signed char, unsigned char, wchar_t, char8_t, char16_t, char32_t |
| 정수형      | short, int, long, long long과 각각의 unsigned 형식                           |
| 부동소수점형   | float, double, long double                                             |
| 빈 타입     | void                                                                   |
| 널 포인터 타입 | std::nullptr_t                                                         |

char, short, int, long, float, double과 같은 기존 기본형은 C와 같은 역할로 사용한다. 정확한 크기는 컴파일러와 실행 환경에 따라 달라질 수 있으며, 같은 플랫폼과 ABI의 C와 C++에서는 대응하는 기본형이 같은 데이터 모델을 사용한다.

타입의 크기, 정렬과 표현 범위를 확인하는 방법은 다음 장에서 자세히 살펴본다.

#### 1.3.2 정수형과 리터럴 접미사

C++의 정수형은 C에서 사용하던 short, int, long을 계승하며 long long을 표준 정수형으로 제공한다.

```
short smallValue = 10;
int value = 100;
long largeValue = 1000L;
long long veryLargeValue = 10000LL;
```

정수 리터럴은 접미사에 따라 타입이 달라질 수 있다.

| 접미사      | 리터럴 타입             | 예      |
|----------|--------------------|--------|
| 없음       | 값을 표현할 수 있는 표준 정수형 | 100    |
| U 또는 u   | 부호 없는 정수형          | 100U   |
| L 또는 l   | long               | 100L   |
| LL 또는 ll | long long          | 100LL  |
| UL, LU   | unsigned long      | 100UL  |
| ULL, LLU | unsigned long long | 100ULL |

소문자 l은 숫자 1과 구분하기 어려우므로 long과 long long 리터럴에는 대문자 L을 사용하는 편이 좋다.

정수형은 부호 있는 형식과 부호 없는 형식으로 나뉜다.

```
int signedValue = -10;
unsigned int unsignedValue = 10U;
```

부호 없는 정수형은 음수를 표현하지 않는 대신 같은 크기의 부호 있는 정수형보다 더 큰 양수를 표현할 수 있다. 그러나 일반적인 계산에 무조건 **unsigned**를 사용하면 부호 있는 정수형과의 비교나 뺄셈에서 예상하지 못한 결과가 발생할 수 있다. 일반적인 수치 계산에는 부호 있는 정수형을 사용하고, 비트 연산이나 크기처럼 음수가 의미 없는 값을 나타낼 때 부호 없는 정수형을 목적에 맞게 사용한다.

**2진수 접두어 0b와 자릿수 구분(')**  
C++14부터 정수 리터럴 앞에 0b 또는 0B를 붙여 2진수로 작성할 수 있다.

```
int flags = 0b1010;
unsigned int mask = 0B11110000U;
```

긴 숫자는 작은따옴표를 자릿수 구분자로 사용할 수 있다. 자릿수 구분자는 값에 영향을 주지 않는다.

```
int binaryValue = 0b1010'1100;
int decimalValue = 1'000'000;
```

2진수 리터럴은 값을 2진수 표기로 작성하는 문법이다. 값을 실제 비트 형태로 출력하는 방법은 2장의 출력 형식 지정에서 살펴본다. 정확한 비트 폭이 필요한 경우에는 `<cstdint>`의 고정 폭 정수형을 사용할 수 있다.

```
#include <cstdint>
std::int32_t id = 100;
std::uint64_t fileSize = 5000000000ULL;
```

`std::int32_t`는 정확히 32비트인 부호 있는 정수형이고, `std::uint64_t`는 정확히 64비트인 부호 없는 정수형이다. 이러한 타입은 해당 비트 폭을 정확하게 표현할 수 있는 정수형이 실행 환경에 존재할 때 정의된다.

고정 폭 정수형은 파일 형식, 네트워크 패킷과 바이너리 데이터처럼 비트 폭이 정확하게 정해져야 하는 경우에 유용하다.

Visual C++의 오래된 코드에서는 64비트 정수를 나타내기 위해 Microsoft 전용 타입인 `__int64`를 사용하기도 했다. `__int64`는 비표준 확장 타입으로, 다른 컴파일러에서는 지원되지 않을 수 있다.

```
__int64 legacyValue = 100;           // Microsoft 전용 비표준 타입
long long standardValue = 100LL;     // 표준 C++ 타입
std::int64_t fixedValue = 100;       // 정확히 64비트인 정수형
```

`long long`은 C++11부터 표준 C++의 기본 정수형으로 정식 채택되었으며 최소 64비트의 표현 범위를 보장한다. 정확히 64비트인 정수형이 필요한 경우에, `<cstdint>`에 정의된 `std::int64_t` 또는 `std::uint64_t`를 사용한다. 새로 작성하는 코드에서는 표준 정수형인 `long long` 또는 고정 폭 정수형을 사용하고, `__int64`는 오래된 Visual C++ 코드를 이해할 때 알아두면 된다.

<정수형과 고정폭 정수형>

```
#include <cstdint>
#include <iostream>
#include <limits>
```

```
using namespace std;

int main()
{
    int signedValue = 0xFFFFFFFF;
    unsigned int unsignedValue = 0xFFFFFFFFU;
    long largeValue = 1000000L;
    long long veryLargeValue = 0x7FFFFFFFFFFFFFFFLL;
    std::int32_t id = 100;
    std::uint64_t fileSize = 5000000000ULL;

    cout << "signedValue(int)          : " << signedValue << '\n';
    cout << "unsignedValue(unsigned int): " << unsignedValue << '\n';
    cout << "largeValue(long)           : " << largeValue << '\n';
    cout << "veryLargeValue(long long) : " << veryLargeValue << '\n';
    cout << "id(int32_t)              : " << id << '\n';
    cout << "fileSize(uint64_t)       : " << fileSize << "\n\n";

#ifdef _MSC_VER
    cout << "__int64(Microsoft type)  -----\\n";
    __int64 legacyValue = 100;
    cout << "sizeof(__int64)         : " << sizeof(__int64) << "byte\\n";
    cout << "legacyValue(__int64)    : " << legacyValue << "\\n\\n";
#endif
}
```

### 1.3.3 논리형(bool)

C++는 참과 거짓을 표현하기 위해 **bool** 타입을 제공한다. **bool** 타입에 저장할 수 있는 논리값은 **true**와 **false**이다.

```
bool isRunning = true;
bool isFinished = false;
```

초기의 C언어에는 참과 거짓을 나타내는 별도의 논리형이 없었다. 따라서 조건이나 상태를 표현할 때 주로 정수형을 사용했다.

```
int isRunning = 1;    // 참
int isFinished = 0;   // 거짓
```

```
bool first = 0;       // false
bool second = 10;     // true
bool third = -1;      // true
```

정수값이 **bool**로 변환되면 원래 정수값이 그대로 저장되는 것이 아니라 **false** 또는 **true**가 저장된다.

```
int value = 10;
bool result = value; // true
```

반대로 **bool** 값이 정수형으로 변환되면 **false**는 0, **true**는 1이 된다.

```
int firstValue = false; // 0
int secondValue = true;  // 1
```

비교 연산자와 논리 연산자의 결과도 **bool** 타입이다.

```
int hp = 50;
bool isAlive = hp > 0;
bool isDead = hp <= 0;
bool canContinue = isAlive && !isFinished;
```

무한 반복 도 정수 1보다 논리값 **true**를 사용하여 의도를 표현할 수 있다.

```
while (true)
{
    // 반복 처리
}
```

**while (1)**도 C++에서 정상적으로 동작하지만, 이 교재에서는 조건이 항상 참이라는 뜻을 직접 나타내기 위해 **while (true)**를 사용한다.



<논리형과 값 변환>

```
#include <iostream>
using namespace std;
int main()
{
    int cStyleRunning = 1;
    bool cppStyleRunning = true;

    bool first  = 0;
    bool second = 10;
    bool third  = -1;

    cout << std::boolalpha;    // bool 값을 1,0 대신, true, false 로 출력.

    cout << "C   방식 논리 상태: " << cStyleRunning << '\n';
    cout << "C++ 방식 논리 상태: " << cppStyleRunning << "\n\n";

    cout << "first  : " << first   << '\n';
    cout << "second : " << second << '\n';
    cout << "third  : " << third  << "\n\n";
}
```

cout은 기본적으로 bool 값을 0과 1로 출력한다. 예제에서는 std::boolalpha를 사용하여 false와 true로 출력하도록 설정했다. 출력 형식과 서식 조정자는 입출력 형식 지정을 다루는 단원에서 살펴본다.

1.3.4 확장된 문자형과 리터럴 접두사

C++는 일반 문자형인 char 외에도 와이드 문자와 Unicode 코드 단위를 표현하기 위한 문자형을 제공한다.

```
char asciiCharacter = 'A';
wchar_t wideCharacter = L'A';
char8_t utf8Character = u8'A';
char16_t utf16Character = u'A';
char32_t utf32Character = U'A';
```

문자 리터럴 앞에 붙는 접두사에 따라 리터럴의 타입이 달라진다.

| 표기    | 타입       | 용도           |
|-------|----------|--------------|
| 'A'   | char     | 일반 문자 코드 단위  |
| L'A'  | wchar_t  | 와이드 문자 코드 단위 |
| u8'A' | char8_t  | UTF-8 코드 단위  |
| u'A'  | char16_t | UTF-16 코드 단위 |
| U'A'  | char32_t | UTF-32 코드 단위 |

char, signed char, unsigned char는 서로 구분되는 타입이다.

```
char character = 'A';
signed char signedValue = -10;
unsigned char unsignedValue = 200;
```

일반 char가 부호 있는 타입처럼 동작하는지 부호 없는 타입처럼 동작하는지는 실행 환경에 따라 달라진다. 작은 정수값의 부호를 명확하게 나타내려면 signed char 또는 unsigned char를 사용한다. wchar\_t의 크기와 인코딩 방식은 실행 환경에 따라 달라질 수 있다. char8\_t는 UTF-8, char16\_t는 UTF-16, char32\_t는 UTF-32의 코드 단위를 나타낸다. 이 타입들은 화면에 표시되는 문자 하나보다 문자 인코딩을 구성하는 코드 단위를 저장한다고 이해하는 것이 정확하다. 하나의 문자는 인코딩 방식에 따라 여러 코드 단위로 표현될 수 있다.

문자형의 크기와 UTF-8·UTF-16·UTF-32의 코드 단위 수는 문자열을 다루는 단원에서 살펴본다.



1.3.5 부동소수점형의 종류와 접미사

부동소수점형은 소수점이 있는 실수를 표현하는 타입이다. C++는 float, double, long double을 제공한다.

```
float speed = 3.5f;
double distance = 1250.75;
long double extendedPi = 3.1415926535897932384626433832795L;
```

부동소수점 리터럴은 접미사에 따라 타입이 결정된다.

| 접미사    | 리터럴 타입      | 예    |
|--------|-------------|------|
| 없음     | double      | 3.5  |
| f 또는 F | float       | 3.5f |
| l 또는 L | long double | 3.5L |

접미사가 없는 부동소수점 리터럴은 기본적으로 double 타입이다. 소문자 l은 숫자 1과 구분하기 어려우므로 long double 리터럴에는 대문자 L을 사용하는 편이 좋다.

매우 크거나 작은 값은 지수 표기법으로 작성할 수 있다.

```
double lightSpeed = 2.99792458e8;
double epsilon = 1.0e-6;
float smallValue = 1.0e-3f;
```

float, double, long double의 정확한 크기와 정밀도는 실행 환경에 따라 달라질 수 있다. double은 float 이상의 정밀도와 표현 범위를 제공하고, long double은 double 이상의 정밀도와 표현 범위를 제공한다. 일부 환경에서는 long double과 double이 같은 크기와 정밀도를 사용할 수 있다.

부동소수점형은 한정된 비트로 실수를 표현하므로 일부 값은 정확한 값이 아니라 근사값으로 저장된다. 부동소수점 오차와 실수값의 비교 방법은 다음 장에서 자세히 살펴본다.

1.3.6 void, nullptr, std::nullptr\_t

void는 값이 없음을 나타내는 타입이다. 함수의 반환 타입에 void를 사용하면 호출한 곳으로 값을 반환하지 않는다는 뜻이다.

```
void PrintMessage()
{
    std::cout << "Hello" << '\n';
}
```

반환 타입이 void인 함수에서도 함수의 실행을 중간에 끝내기 위해 return을 사용할 수 있다. 이때는 반환값을 작성하지 않는다.

```
void PrintPositive(int value)
{
    if(value < 0)
        return;
    std::cout << value << '\n';
}
```

C++에서는 매개변수가 없는 함수를 빈 괄호 ()로 선언한다.

```
void PrintMessage();
```

C17까지의 C에서는 빈 괄호 ()가 매개변수의 개수와 타입을 지정하지 않았다는 뜻이었으며, 매개변수가 없음을 명확히 나타내려면 (void)를 사용했다.

| 표기             | C17까지의 C         | C++      |
|----------------|------------------|----------|
| Function()     | 매개변수 정보가 지정되지 않음 | 매개변수가 없음 |
| Function(void) | 매개변수가 없음         | 매개변수가 없음 |

C와의 문법적 호환을 위해 C++에서도 (void) 표기를 사용할 수 있지만, C++에서는 빈 괄호 ()를 사용한다. C23부터 C에서도 빈 괄호 ()가 매개변수가 없는 함수를 의미한다. 그러나 기존 C 코드와 C17까지의 문법으로 작성된 코드를 이해하려면 두 표기의 차이를 알아둘 필요가 있다.

void 타입의 객체나 배열을 만들 수 없다. void는 객체에 저장할 값이나 객체의 크기를 정의하지 않기 때문이다.

```
// void value;           // 오류
// void values[10];      // 오류
```

다만 void\*는 사용할 수 있다. void\*는 가리키는 객체의 구체적인 타입 정보가 정해지지 않은 포인터이다.

```
int value = 100;
void* memory = &value;
```

void\*에는 가리키는 객체의 타입 정보가 없으므로 값을 직접 읽거나 변경할 수 없다. 값을 사용하려면 원래 객체 포인터 타입으로 변환해야 한다.

C와 기존 C++ 코드에서는 널 포인터를 나타내기 위해 정수 0이나 NULL을 사용하기도 했다.

```
int* firstPointer = 0;
int* secondPointer = NULL;
```

그러나 0은 정수 리터럴이고, NULL도 구현에 따라 정수 형태의 매크로로 정의될 수 있다. C++11부터는 널 포인터를 명확하게 표현하기 위해 nullptr를 제공한다.

```
int* pointer = nullptr;
```

새로 작성하는 C++ 코드에서는 0이나 NULL 대신 nullptr를 사용한다.

```
int* intPointer = nullptr;
double* doublePointer = nullptr;
```

nullptr의 타입은 <cstdlib>에 정의된 std::nullptr\_t이다.

```
#include <cstdlib>
std::nullptr_t nullValue = nullptr;
```

일반적인 프로그램에서 std::nullptr\_t 타입의 변수를 직접 선언하는 경우는 많지 않다. 주로 함수 오버로딩이나 템플릿 코드에서 nullptr 자체를 구분할 때 사용한다. 이에 대한 특징과 내용은 함수 오버로딩을 다루는 단원에서 자세히 살펴본다.

<void와 nullptr 확인>

```
#include <iostream>
using namespace std;

void PrintMessage(/*매개 변수 없으면 void 안씀. 빈 괄호 유지*/)
{
    cout << "반환값이 없는 함수입니다.\n";
}

int main()
{
    PrintMessage();

    int value = 100;
    void* memory = &value;

    cout << "객체의 주소: " << memory << '\n';

    int* pointer = nullptr;
    if(pointer == nullptr)
        cout << "pointer는 널 포인터입니다.\n";
}
```

## 1.4 C++에서 추가된 편리한 문법과 기능

C++는 C의 기본 문법을 대부분 그대로 사용할 수 있으면서, 같은 작업을 더 간결하고 명확하게 표현할 수 있는 문법과 표준 라이브러리 기능을 추가하였다. 기존 방식이 사라진 것이 아니라 필요에 따라 선택할 수 있는 새로운 작성 방법이 더해진 것이다.

C++는 오랜 기간에 걸쳐 표준이 개정되었으며, C++11 이후에는 대체로 3년 주기로 새로운 표준이 발표되고 있다. 따라서 회사와 프로젝트마다 사용하는 C++ 버전이 다를 수 있다. 이 교재는 C++20을 기준으로 하며, C++98 이후에 추가된 기능은 제목이나 본문에 최초로 표준화된 버전을 표시한다.

이 절에서는 C에서 자주 사용하던 동적 메모리 할당, 제어문, 반복문과 난수 생성을 C++에서는 어떻게 작성할 수 있는지 살펴본다.

### 1.4.1 {new, delete}, {new[], delete[]} 를 이용한 동적 메모리 할당

C에서는 malloc() 함수로 필요한 크기의 메모리를 할당하고 free() 함수로 해제한다. 할당한 메모리를 0으로 초기화하려면 memset() 함수를 별도로 호출한다.

```
// C 방식의 동적 메모리 할당과 해제
#include <stdlib.h>
#include <string.h>

void MemoryAllocFree(void)
{
    char* memory = malloc(100 * sizeof(char));
    if (memory != NULL)
    {
        memset(memory, 0, 100);
        free(memory);
    }
}
```

malloc()은 지정한 바이트 수만큼 메모리를 할당하고 void\*를 반환한다. C에서는 void\*를 다른 객체 포인터 타입에 암시적으로 대입할 수 있으므로 별도의 형 변환이 필요하지 않다.

C++에서도 C 표준 라이브러리의 malloc(), memset()과 free()를 사용할 수 있다. C++는 <stdlib.h>, <string.h>와 같은 C 헤더도 지원하지만, C++ 코드에서는 C 표준 라이브러리 헤더에 c를 붙인 <cstdlib>와 <cstring>을 사용한다.

```
// C++에서 C 라이브러리를 사용한 동적 메모리 할당과 해제
#include <cstdlib>
#include <cstring>
using namespace std;

void MemoryAllocFree()
{
    char* memory = static_cast<char*>(malloc(100 * sizeof(char)));
    if(memory != nullptr)
    {
        memset(memory, 0, 100);
        free(memory);
    }
}
```

C++에서는 void\*를 다른 객체 포인터 타입에 암시적으로 대입할 수 없으므로 static\_cast를 이용한 명시적인 형 변환이 필요하다. 또한 널 포인터는 NULL 대신 nullptr(C++11)로 표현한다.

C++에서는 new, new[], delete, delete[] 연산자를 사용하여 객체와 배열을 동적으로 생성하고 해제할 수 있다. malloc(), memset()과 free()는 라이브러리 함수이므로 관련 헤더 파일을 포함해야 하지만, new와 delete는 C++ 언어가 제공하는 연산자이므로 기본적인 사용에는 별도의 헤더 파일이 필요하지 않다.

다음 코드는 char 타입 객체 100개를 동적으로 생성하고 모든 원소를 0으로 초기화하는 예제이다. new char[100]()은 char 객체 100개를 동적으로 생성하고 모든 원소를 0으로 값 초기화한 뒤 첫 번째 원소의 주소를 반환한다. 따라서 C 코드처럼 memset()을 별도로 호출할 필요가 없다.

```
// new[]와 delete[]를 사용한 동적 메모리 할당과 해제
void MemoryNewDelete()
{
    // new[] 로 객체 배열을 생성하고, ()를 사용하여 모든 배열 요소를 0으로 초기화.
    char* memory = new char[100]();

    // ... 메모리 사용 ...

    // 메모리 사용 후 delete[]로 해제.
    delete[] memory;
}
```

기본 `new`는 메모리를 할당하지 못하면 `std::bad_alloc` 예외를 발생시킨다. 할당 실패를 예외 대신 `nullptr`로 확인하려면 `<new>`에 정의된 `std::nothrow`를 사용한다.

```
#include <new>
// std::nothrow를 적용, 할당 실패를 nullptr로 확인
void MemoryNewDeleteNothrow()
{
    char* memory = new (std::nothrow) char[100]();
    if(memory != nullptr)
    {
        delete[] memory;
    }
}
```

`std::nothrow`를 사용한 `new`는 메모리 할당에 실패하면 `nullptr`를 반환한다. `new`의 할당 실패 처리와 `std::nothrow`의 자세한 사용 방법은 `new` 연산자 단원에서 살펴본다.

`new` 연산자로 생성한 단일 객체는 `delete`로 해제하고, `new[]`로 생성한 객체 배열은 `delete[]`로 해제한다.

```
#include <new>
// 단일 객체
int* single = new (std::nothrow) int(200);
if (single != nullptr)
{
    delete single;
}

// 객체 배열
int* values = new (std::nothrow) int[1000]();
if (values != nullptr)
{
    delete[] values;
}
```

`new int(200)`은 `int` 객체 하나를 동적으로 생성하고 값 200으로 초기화한다. `new int[1000]()`은 `int` 객체 1000개를 동적으로 생성하고 모든 원소를 0으로 값 초기화한다.

동적 할당과 해제 짝 맞추기

동적 할당과 해제는 반드시 서로 대응하는 방식을 사용해야 한다. `new`로 생성한 단일 객체는 `delete`, `new[]`로 생성한 객체 배열은 `delete[]`로 해제하며, `malloc()`, `calloc()`, `realloc()`으로 할당한 메모리는 `free()`로 해제한다.

| 동적 할당   |                                                                        | 해제                  |
|---------|------------------------------------------------------------------------|---------------------|
| 단일 객체   | <b>new</b>                                                             | <b>delete</b>       |
| 객체 배열   | <b>new[]</b>                                                           | <b>delete[]</b>     |
| C 방식 함수 | <code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> | <code>free()</code> |

`new`와 `free()`, `malloc()`과 `delete`처럼 서로 다른 방식을 조합해서는 안 된다. `malloc()`으로 할당한 메모리는 `free()`로 해제하고, `new`와 `new[]`로 생성한 객체는 각각 `delete`와 `delete[]`로 해제한다.

```
int* value = new int(10);
// free(value); // 잘못된 사용
delete value;

int* values = new int[10]();
// delete values; // 잘못된 사용
delete[] values;
```

`new`와 `delete`는 C++의 객체 생성과 수명 관리에 연결되는 기본 기능이다. 클래스 객체를 `new`로 생성하면 생성자가 호출되고, `delete`로 해제하면 소멸자가 호출된다. 생성자와 소멸자는 클래스 단원에서 자세히 살펴본다.  
새로 작성하는 일반적인 C++ 코드에서는 원시 `new`와 `delete`를 직접 관리하기보다 `std::vector`와 같은 표준 컨테이너, 또는 스마트 포인터를 우선 사용한다. 자세한 사용 방법은 이후 단원에서 살펴본다.

### 1.4.2 제어문에서 변수 선언하기

C에서는 일반적으로 제어문에서 사용할 변수를 먼저 선언하고, 그다음 조건을 검사한다.

```
int value = GetValue();
if (value > 0)
{
    std::cout << value << '\n';
}
```

C++에서는 `if`, `switch`, `while`의 조건에서 변수를 선언하여 변수의 사용 범위를 제어문 안으로 제한할 수 있다. 또한 C++17부터 `if`와 `switch`에 초기화 구문을 별도로 작성할 수 있다.

조건문에서의 선언(**Declaration in condition - C++98**)  
`if`문의 조건 자리에 변수를 선언할 수 있다.

```
if (int value = GetValue())
{
    std::cout << value << '\n';
}
```

`GetValue()`가 반환한 값으로 `value`를 초기화한 뒤, `value`를 `bool`로 변환한 결과가 참이면 본문을 실행한다. 정수형에서는 0이 거짓이고 0이 아닌 값은 참이다.

포인터를 반환하는 함수에도 당연히 사용할 수 있다.

```
if (Character* character = FindCharacter())
{
    character->Update();
}
```

`FindCharacter()`가 유효한 포인터를 반환하면 본문을 실행하고, `nullptr`를 반환하면 실행하지 않는다.

조건에서 선언한 변수는 해당 제어문 안에서만 사용할 수 있다. `if`문에서 선언한 변수는 연결된 `else`문에서도 사용할 수 있다.

```
if (int value = GetValue())
{
    std::cout << "값: " << value << '\n';
}
else
{
    // else 문에서도 사용가능
    std::cout << "반환된 값: " << value << '\n';
}

// if-else 문의 범위를 value는 여기서 사용할 수 없음
// std::cout << "value: " << value << '\n';
```

초기자가 있는 `if` 문(**If statement with initializer - C++17**)

C++17부터는 `if`문의 조건 앞에 초기화 구문을 작성할 수 있다. 문법은 다음과 같다.

```
if (초기화 구문; 조건식)
```

먼저 초기화 구문을 실행하고, 이어서 조건식을 검사한다. 초기화 구문에서 선언한 변수는 조건식, `if` 본문과 연결된 `else`문에서 사용할 수 있다.

```
if (int value = GetValue(); value > 0)
{
    std::cout << "양수: " << value << '\n';
}
else
{
    std::cout << "0 또는 음수: " << value << '\n';
}
```

변수를 제어문 밖에 선언하지 않으므로 필요 이상으로 넓은 범위에서 이름이 유지되는 것을 막을 수 있다.

초기자가 있는 **switch**문(**Switch statement with initializer - C++17**)

switch문에도 초기화 구문을 작성할 수 있다. 문법은 다음과 같다.

```
switch (초기화 구문; 조건식)
```

초기화 구문에서 선언한 **command**는 switch문의 조건과 모든 **case** 영역에서 사용할 수 있다.

```
switch (int command = GetCommand(); command)
{
case 1:
    std::cout << "시작\n";
    break;
case 2:
    std::cout << "종료\n";
    break;
default:
    std::cout << "알 수 없는 명령\n";
    break;
}
```

**while**문에서 조건식 내 선언(**Declaration in condition - C++98**)

while문도 조건 자리에 변수를 선언할 수 있다.

```
while (int value = GetReadValue())
{
    std::cout << value << '\n';
}
```

반복할 때마다 **ReadValue()**의 반환값으로 **value**를 초기화하고 조건을 검사한다. 값이 0이 되면 반복을 끝낸다.

포인터를 차례로 얻는 코드에도 자주 사용된다.

```
while (Node* node = GetNextNode())
{
    node->Process();
}
```

**GetNextNode()**가 **nullptr**를 반환할 때 반복이 끝난다.

**while** 문 초기자를 아직까지 지원이 안된다. 따라서 다음 문법은 사용할 수 없다.

```
// 오류
// while (int value = GetReadValue(); value > 0)
// {
// }
```

**while**문에서 초기화와 별도의 조건식이 모두 필요하다면 **C** 방식으로 반복문 밖에서 변수를 선언하거나 반복문 안에서 값을 갱신하는 방식으로 작성한다.

```
int value = GetReadValue();
while (value > 0)
{
    std::cout << value << '\n';
    value = GetReadValue();
}
```

제어문 안의 변수 선언은 새로운 계산 기능을 만드는 문법이 아니라 변수의 사용 범위를 필요한 영역으로 제한하여 코드를 읽고 관리하기 쉽게 만드는 기능이다.

### 1.4.3 범위 기반 **for**문 (**Range - for**)

**C**에서 배열의 모든 원소를 순회하려면 인덱스를 이용하는 일반 **for**문을 사용한다.

```
int values[5] = { 10, 20, 30, 40, 50 };
for (int index = 0; index < 5; ++index)
```



```
{
    std::cout << values[index] << '\n';
}
```

C++에서도 이 형태를 그대로 사용할 수 있다. 인덱스가 필요하거나 일부 원소만 순회해야 한다면 일반 **for**문이 적합하다.

C++에는 배열이나 컨테이너의 모든 원소를 차례로 처리하기 위한 범위 기반 **for**문이 추가되었다.

범위 기반 **for**문(**Range-based for statement - C++11**)

범위 기반 **for**문의 기본 문법은 다음과 같다.

```
for (원소 선언 : 범위)
```

배열의 원소를 차례로 출력할 수 있다.

```
int values[] = { 10, 20, 30, 40, 50 };
for (int value : values)
{
    std::cout << value << '\n';
}
```

**value**에는 배열의 각 원소가 차례로 복사된다. 배열의 크기와 인덱스를 직접 작성하지 않으므로 모든 원소를 순회한다는 의도가 분명하게 드러난다.

**std::vector**와 같은 표준 컨테이너에도 같은 문법을 사용할 수 있다.

```
#include <vector>
std::vector<int> values = {10, 20, 30, 40, 50};
for (int value : values)
{
    std::cout << value << '\n';
}
```

범위 기반 **for**문에서 원소를 값으로 받으면 원본 원소가 복사된다. 참조를 이용해 원본을 변경하거나 복사 비용을 줄이는 방법은 참조 단원에서 자세히 살펴본다.

초기화 구문이 있는 범위 기반 **for**문(**C++20**)

C++20부터 범위 기반 **for**문 앞부분에 초기화 구문을 작성할 수 있다.

```
for (초기화 구문; 원소 선언 : 범위)
```

예를 들어 함수가 반환한 컨테이너를 지역 변수에 저장한 뒤 바로 순회할 수 있다.

```
for (std::vector<int> values = GetValues(); int value : values)
{
    std::cout << value << '\n';
}
```

**values**는 반복문을 시작하기 전에 한 번 초기화되며, 범위 기반 **for**문 전체에서 사용할 수 있다. 반복문이 끝나면 **for** 문 안에 선언한 **values**의 수명도 끝난다.

초기화 구문에는 반복에 필요한 추가 값을 선언할 수도 있다.

```
for (int index = 0; int value : values)
{
    std::cout << index << ": " << value << '\n';
    ++index;
}
```

여기서 **index**는 원소를 순회할 때마다 자동으로 증가하지 않으므로 반복문 본문에서 직접 증가시킨다.

C의 일반 **for**문을 기준으로 보면 C++에서는 다음 세 가지 형태를 사용할 수 있다.

```
// C와 C++의 일반 for문
for (초기화; 조건; 증감)
```



```

{
}
// C++11 범위 기반 for문
for (원소 선언 : 범위)
{
}
// C++20 초기화 구문이 있는 범위 기반 for문
for (초기화 구문; 원소 선언 : 범위)
{
}

```

세 번째 형태는 완전히 별개의 반복문이라기보다 C++11의 범위 기반 for문에 초기화 구문이 추가된 형태이다. 그러나 코드를 작성할 때는 기존 일반 for문 외에 두 가지 범위 기반 작성 형태를 선택할 수 있다.

<범위 기반 for문 확인>

```

#include <iostream>
#include <vector>
using std::cout;

std::vector<int> GetValues()
{
    std::vector<int> values = {11, 22, 33, 44, 55};
    return values;
}

int main()
{
    cout << "C++11 범위 기반 for문\n";
    int values[] = {10, 20, 30, 40, 50};
    for(int value : values)
    {
        cout << value << ' ';
    }

    cout << "\n\n";
    cout << "C++20 초기화 구문이 있는 범위 기반 for문\n";
    for(std::vector<int> dynamicValues = GetValues();
        int value : dynamicValues)
    {
        cout << value << ' ';
    }
    cout << '\n';
}

```

#### 1.4.4 C++ 난수 라이브러리(C++11)

C에서는 rand()로 난수를 얻고 srand()로 난수열의 시작값인 시드를 지정한다.

```

#include <cstdlib>
#include <ctime>

int range = 1000;
std::srand(static_cast<unsigned int>(std::time(nullptr)));
int rnd = std::rand() % (range + 1);

```

이 코드는 0부터 range까지의 값을 얻기 위해 % 연산을 사용한다. 그러나 rand()의 품질과 표현 범위는 실행 환경에 따라 달라질 수 있고, % 연산으로 범위를 제한하면 각 값이 정확히 같은 확률로 나오지 않을 수 있다.

C++11부터 <random> 헤더에 난수 엔진과 분포가 추가되었다.

```

#include <random>
std::mt19937 engine(100);
std::uniform_int_distribution<int> distribution(0, 99);
int value = distribution(engine);

```

C++ 난수 라이브러리는 난수 생성을 두 역할로 나눈다.

| 구성 요소 | 역할                        |
|-------|---------------------------|
| 난수 엔진 | 일정한 규칙에 따라 원시 난수열 생성      |
| 분포    | 엔진의 결과를 원하는 범위와 확률 분포로 변환 |

`std::mt19937`은 메르센 트위스터 알고리즘을 사용하는 난수 엔진이며, `mt19937_64` 64비트 난수 엔진이다. 생성자에 전달된 값은 시드이다.

```
std::mt19937 engine(100);
```

같은 엔진과 같은 시드를 사용하면 같은 난수열이 만들어진다. 게임의 오류를 재현하거나 테스트 결과를 반복해야 할 때 유용하다.

실행할 때마다 다른 시드값을 얻기 위해 `std::random_device`를 사용할 수 있다.

```
std::random_device seed;
std::mt19937 engine(seed());
```

`std::uniform_int_distribution<int>`는 지정한 범위의 정수가 균등한 확률로 나오도록 변환한다.

```
std::uniform_int_distribution<int> dice(1, 6);
int result = dice(engine);
```

정수 분포의 양쪽 끝값은 모두 포함되므로 결과는 1부터 6까지이다. 실수 범위에는 `std::uniform_real_distribution`을 사용할 수 있다.

```
std::uniform_real_distribution<double> distribution(0.0, 1.0);
double value = distribution(engine);
```

난수 엔진은 값을 생성할 때마다 내부 상태가 바뀐다. 따라서 난수가 필요할 때마다 엔진을 새로 생성하거나 같은 시드로 다시 초기화하면 같은 값이 반복될 수 있다.

```
std::mt19937 engine(100);
for (int count = 0; count < 5; ++count)
{
    int value = distribution(engine);

    std::cout << value << '\n';
}
```

하나의 엔진을 유지하면서 필요한 분포 객체에 전달하는 방식으로 사용한다. 게임에서 같은 결과를 재현해야 한다면 고정된 시드를 사용하고, 실행할 때마다 다른 결과가 필요하다면 실행 환경에서 얻은 시드값을 사용할 수 있다.

<C++ 난수 생성>

```
Snippet
#include <iostream>
#include <iomanip>
#include <random>
using namespace std;

int main()
{
    std::random_device seed;
    std::mt19937 engine(seed());

    cout << "std::mt19937: [주사위 랜덤]\n";
    std::uniform_int_distribution<int> dice(1, 6);
    for(int count = 0; count < 10; ++count)
    {
        cout << dice(engine) << ' ';
    }
    cout << "\n\n";

    std::mt19937_64 engine64(seed());
```

```
cout << "std::mt19937_64: [0, 99]\n";
std::uniform_int_distribution<int> percent(0, 99);
for(int count = 0; count < 10; ++count)
{
    cout << percent(engine64) << ' ';
}
cout << "\n\n";

cout << "std::mt19937_64: [0.0 , 1.0]\n";
cout << fixed << setprecision(7);
std::uniform_real_distribution<double> distribution(0.0, 1.0);
for(int count = 0; count < 10; ++count)
{
    cout << distribution(engine64) << '\n';
}
cout << '\n';
}
```

1.4.5 C++ 버전별 정리

| 기능                           | 표준 버전 |
|------------------------------|-------|
| new, delete, new[], delete[] | C++98 |
| 조건에서 변수 선언                   | C++98 |
| if와 switch의 초기화 구문           | C++17 |
| 범위 기반 for문                   | C++11 |
| 초기화 구문이 있는 범위 기반 for문        | C++20 |
| <random> 난수 라이브러리            | C++11 |

C++ 프로젝트의 언어 표준이 낮게 설정되어 있으면 최신 문법을 지원하는 컴파일러에서도 해당 기능을 사용할 수 없다. 다른 회사나 기존 프로젝트의 코드를 작성할 때는 프로젝트가 사용하는 C++ 표준 버전을 먼저 확인해야 한다.

1.4.5 std::sort()를 이용한 정렬

C에서는 배열의 원소를 정렬하려면 정렬 알고리즘을 직접 작성하거나 qsort() 함수를 사용한다. C++에서는 <algorithm>에 정의된 std::sort()를 이용해 배열이나 컨테이너의 원소를 정렬할 수 있다.

std::sort()에는 정렬할 범위의 시작 위치와 마지막 원소 다음 위치를 전달한다.

```
std::sort(시작 위치, 마지막 원소 다음 위치);
```

다음 코드는 정수 배열을 오름차순으로 정렬한다.

```
int values[] = {1, 9, 2, 3, 5, 7, 4, 17};
std::sort(values, values + 8);
```

values는 배열의 첫 번째 원소를 가리키고, values + 8은 마지막 원소 다음 위치를 가리킨다. 원소 개수를 직접 작성하면 배열의 크기가 변경될 때 함께 수정해야 한다.

C++20의 std::span을 사용하면 배열의 원소를 복사하지 않고 배열 전체를 하나의 범위로 표현할 수 있다. 배열로부터 std::span을 생성하면 배열의 크기가 자동으로 결정되며, begin(), end(), size()를 이용할 수 있다.

```
span<int> range(values);
std::sort(range.begin(), range.end());
```

std::sort()는 별도의 정렬 기준을 지정하지 않으면 오름차순으로 정렬한다. 내림차순으로 정렬하려면 <functional>에 정의된 std::greater를 세 번째 인수로 전달한다.

```
std::sort(range.begin(), range.end(), std::greater<int>());
// 또는 타입을 생략해도 사용할 수 있다.
std::sort(range.begin(), range.end(), std::greater<>());
```

<std::sort 오름차순 내림차순 정렬>

```
#include <iostream>
#include <algorithm>
#include <span>
using std::span;
using std::cout;

int main()
{
    int values[] = {1, 9, 2, 3, 5, 7, 4, 17};

    cout << "오름 차순 정렬" << '\n';
    span<int> range(values);
    std::sort(range.begin(), range.end());
    for(int v : values)
        cout << v << " ";
    cout << "\n\n";

    cout << "내림 차순 정렬" << '\n';
    span<int> range(values);
    std::sort(range.begin(), range.end(), std::greater<int>());
    for(int v : values)
        cout << v << " ";
    cout << '\n';
}
```

```
#include <iostream>
#include <algorithm>
#include <span>
using std::span;
using std::cout;

int main()
{
    int values[] = {1, 9, 2, 3, 5, 7, 4, 17};
    span<int> range(values);
    std::sort(range.begin(), range.end());
    for(int v : values)
        cout << v << " ";
    cout << '\n';
}
```

실행 결과는 다음과 같다.

```
오름 차순 정렬
1 2 3 4 5 7 9 17

내림 차순 정렬
17 9 7 5 4 3 2 1
```

간단히 기본형으로 구성된 배열의 정렬을 살펴 보았다. 구조체 정렬과 사용자 정의 정렬 기준은 표준 알고리즘 단원에서 자세히 살펴본다.

## 1.5 정리

C++는 C의 변수, 연산자, 제어문, 함수, 배열, 구조체와 포인터를 대부분 계승한다. 따라서 C를 학습한 독자는 기존 지식을 바탕으로 C++를 배울 수 있다. 그러나 C++는 C의 모든 프로그램을 그대로 받아들이는 완전한 상위 호환 언어는 아니며, 일부 타입 변환과 문법 규칙을 C보다 엄격하게 검사한다.

C++에서는 C에서 이어진 키워드와 함께 **namespace**, **class**, **nullptr**, **auto**처럼 새로운 언어 기능을 표현하는 키워드를 사용한다. 키워드는 언어에서 특별한 의미로 예약된 단어이므로 변수나 함수 등의 식별자로 사용할 수 없다.

C++에서는 **cin**, **cout**과 **getline()**을 이용해 값을 입력하고 출력할 수 있다. 변수의 타입에 따라 입출력 동작이 결정되므로 형식 지정자를 직접 작성하지 않으며, 일반 변수에 값을 입력할 때 주소 연산자 **&**를 사용하지 않는다.

C++는 C의 기본형 타입을 대부분 그대로 사용하면서 **bool**, **wchar\_t**, **char8\_t**, **char16\_t**, **char32\_t**, **std::nullptr\_t** 등의 타입을 제공한다. **bool**은 논리 상태를 **true**와 **false**로 직접 표현하며, 널 포인터는 정수 0이나 **NULL**보다 **nullptr**로 표현하는 것이 의도를 명확하게 나타낸다.

정수 리터럴에는 타입을 지정하는 접미사를 사용할 수 있으며, C++14부터는 0b 접두사를 이용해 2진수 리터럴을 작성할 수 있다. 정확한 비트 폭이 필요한 경우에는 <stdint>에 정의된 고정 폭 정수형을 사용할 수 있다.

부동소수점형은 float, double, long double로 구분하며 리터럴 접미사로 타입을 지정할 수 있다. 부동소수점값은 일부 실수를 정확하게 표현하지 못하고 근사값으로 저장하므로 계산과 비교에서 오차를 고려해야 한다.

C++에서는 new, new[], delete, delete[] 연산자를 이용해 객체와 배열을 동적으로 생성하고 해제할 수 있다. new로 생성한 단일 객체는 delete, new[]로 생성한 객체 배열은 delete[]로 해제해야 하며, malloc(), calloc(), realloc()으로 할당한 메모리는 free()로 해제해야 한다.

C++는 제어문 안에서 변수를 선언할 수 있으며, C++17부터는 if와 switch에 초기화 구문을 작성할 수 있다. 변수를 사용하는 제어문 안으로 유효 범위를 제한하면 불필요하게 오랫동안 변수가 유지되는 것을 막을 수 있다.

C++11부터 배열과 컨테이너의 모든 원소를 순회하는 범위 기반 for문을 사용할 수 있으며, C++20부터는 범위 기반 for문에도 초기화 구문을 작성할 수 있다.

C++11의 <random> 라이브러리는 난수 생성을 엔진과 분포로 나누어 처리한다. std::mt19937과 std::uniform\_int\_distribution을 이용하면 원하는 범위의 정수를 생성할 수 있으며, 고정된 시드를 사용하면 같은 난수열을 다시 재현할 수 있다.

C++에서는 <algorithm>에 정의된 std::sort()를 이용해 배열이나 컨테이너의 원소를 정렬할 수 있다. 정렬할 범위의 시작 위치와 마지막 원소 다음 위치를 전달하며, 별도의 정렬 기준을 지정하지 않으면 오름차순으로 정렬한다. C++20의 std::span을 사용하면 배열의 원소를 복사하지 않고 하나의 범위로 표현하여 begin(), end(), size()를 이용할 수 있다.

C++에서는 C 방식의 코드를 그대로 작성할 수도 있지만, 추가된 문법과 표준 라이브러리를 이용하면 변수의 범위, 객체의 수명과 자원의 소유 관계를 더 명확하게 표현할 수 있다.

## 1.6 학습 과제

공통 조건

- 입력에는 cin, 출력에는 cout을 사용한다.
- 난수 생성에는 C++ random을 사용한다. rand()와 srand()는 사용하지 않는다.
- 동적 생성과 해제는 new-delete 만 사용한다.
- 필요한 기능은 적절하게 함수로 분리한다.

문제 1. 랜덤 훌짝 맞추기

게임은 총 5판 3선승으로 진행한다.

컴퓨터가 1부터 100 사이의 정수를 무작위로 생성하고, 사용자가 홀수 또는 짝수를 선택하여 결과를 맞히는 프로그램을 작성한다.

사용자는 매 회 다음과 같이 입력한다.

1. 홀수

2. 짝수

매회 컴퓨터가 생성한 숫자와 사용자의 선택을 비교하여 정답 여부를 출력한다.

3번의 승리 5회의 게임이 끝나면 승패를 출력한다.

문제 2. 업다운 숫자 맞추기

컴퓨터가 10부터 99 사이의 정수 하나를 무작위로 생성한다. 사용자는 최대 7번 안에 정답을 맞혀야 한다.

사용자가 입력한 값이 정답보다 작으면 더 큰 값입니다., 정답보다 크면 더 작은 값입니다.를 출력한다.

정답을 맞히면 시도 횟수를 출력하고 게임을 종료한다.

문제 3. 랜덤 값 저장과 분석

학생 성적에 대한 int형 배열을 10~60 사이 동적으로 생성하되 반드시 new-delete 사용 사용한다.

동적으로 생성된 배열에 랜덤값을 10~99 값을 랜덤으로 채우되, 전부다 다른 값을 채워야 한다.

생성한 결과를 오름 차순으로 정렬하고 모두 출력한다.

또한 값을 계산한다.

- 전체 합계
- 평균값
- 최대값
- 최소값
- 60점 미만인 학생 수
- 60점 이상인 학생 수
- 70점 이상인 학생 수
- 80점 이상인 학생 수
- 90점 이상인 학생 수

가급적 범위 기반 for문을 사용한다.

